

LAB EXPERIMENT REPORT

Object-Oriented Programming using Java

Basic Information	
Experiment No.	_____
Date	_____
Title	Pattern Frequency Counter for Text Files in Java

1. Objective / Aim

To develop a Java program that reads a text file and counts how many times a user-given pattern (word or string) occurs in it, using file handling (BufferedReader, FileReader), console input (Scanner) and string searching with `indexOf()`.

2. System Requirements

Hardware Requirements

- Processor: Intel Core i3 / AMD equivalent or higher
- RAM: 4 GB minimum (8 GB recommended)
- Hard disk: 500 MB free space
- Input/Output: Keyboard, monitor

Software Requirements

- Operating System: Windows 10/11, Linux or macOS
- Java Development Kit (JDK) 8 or higher
- Editor / IDE: Eclipse, IntelliJ IDEA, NetBeans or VS Code
- Command Prompt / Terminal
- A plain text (.txt) input file

3. Algorithm / Logic Description

Key idea: Read the file one line at a time. In each line, repeatedly call `indexOf(pattern, index)` to find the next match. After each match, increase the counter and move `index` forward by the pattern length, so the search continues after the match (matches do not overlap). When `indexOf` returns -1, move to the next line.

Algorithm:

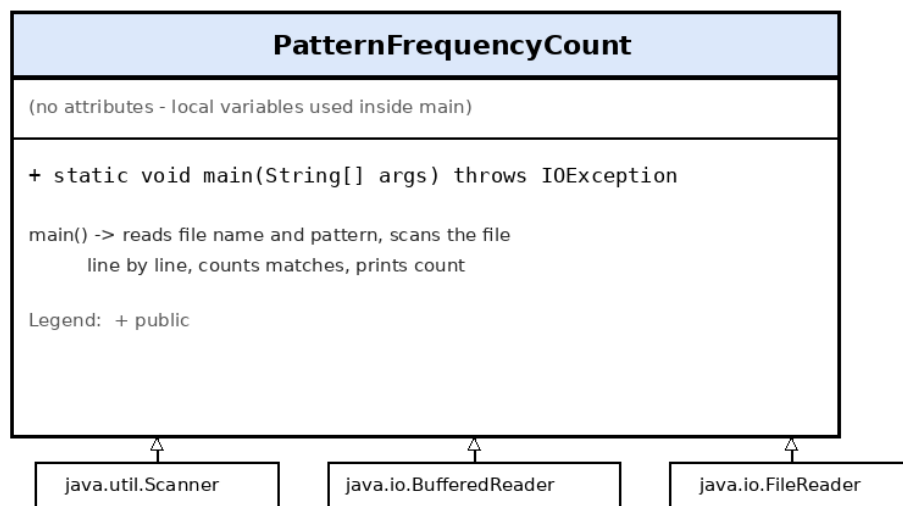
1. Start.
2. Read the file name and the pattern from the user.
3. Open the file using `BufferedReader(FileReader)`.
4. Set `count = 0`.
5. Read the next line. If there are no more lines, go to step 9.

6. Set index = 0.
7. Find index = line.indexOf(pattern, index). If index is -1, go to step 5.
8. Increase count by 1, set index = index + pattern length, and go to step 7.
9. Close the file and display count.
10. Stop.

Notes: The search is case-sensitive ("Java" and "java" are different) and it matches substrings, not whole words (searching "is" also matches the "is" inside "this"). The pattern must not be empty, otherwise the inner loop would never advance. If the file does not exist, a `FileNotFoundException` is thrown.

Complexity: Time $O(N \times M)$ in the worst case, where N is the total number of characters in the file and M is the pattern length; space $O(L)$ for the longest line.

4. UML Class Diagram



5. Source Code

```
import java.io.*;
import java.util.Scanner;

public class PatternFrequencyCount {
    public static void main(String[] args) throws IOException {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter file name: ");
        String fileName = sc.nextLine();

        System.out.print("Enter pattern to search: ");
        String pattern = sc.nextLine();

        BufferedReader br = new BufferedReader(
            new FileReader(fileName));

        String line;
        int count = 0;

        while ((line = br.readLine()) != null) {
            int index = 0;

            while ((index = line.indexOf(pattern, index)) != -1) {
                count++;
                index += pattern.length();
            }
        }

        br.close();

        System.out.println("Pattern frequency: " + count);

        sc.close();
    }
}
```

6. Compilation and Execution Commands

Save the program as **PatternFrequencyCount.java** and keep the input file (e.g., **sample.txt**) in the same folder. Open a terminal in that folder and run:

```
javac PatternFrequencyCount.java
java PatternFrequencyCount
```

When prompted, enter the file name (e.g., `sample.txt`) and then the pattern to search.

7. Output

Contents of sample.txt:

```
Java is a popular programming language.
Java programs run on the Java Virtual Machine.
Learning Java makes object oriented programming easy.
```

```
Terminal - Pattern Frequency Count

$ cat sample.txt
Java is a popular programming language.
Java programs run on the Java Virtual Machine.
Learning Java makes object oriented programming easy.

$ javac PatternFrequencyCount.java
$ java PatternFrequencyCount
Enter file name: sample.txt
Enter pattern to search: Java
Pattern frequency: 4

$ java PatternFrequencyCount
Enter file name: sample.txt
Enter pattern to search: programming
Pattern frequency: 2

$ java PatternFrequencyCount
Enter file name: sample.txt
Enter pattern to search: is
Pattern frequency: 1
$ _
```

Pattern	Expected count	Actual output
Java	4	4
programming	2	2
is	1	1

8. Result / Conclusion

The program successfully counts the number of occurrences of a given pattern in a text file by scanning it line by line with `indexOf()`. The experiment demonstrates file handling, string searching and user input in Java.